DC-GUARDIAN — Installation and User Guide

NUS-ISS Intelligent Reasoning Systems Practice Module · Group 22

Repository: https://github.com/Jowharah/IRS-PM-2026-10-25-AIS-GRP22-DC-Guardian-System · code version 0ae403e

This guide installs DC-GUARDIAN on a new computer and explains how to use it. It is also the "Installation and User Guide" appendix of the project report.

DC-GUARDIAN is a research prototype for multi-domain data-centre monitoring.

- **What it watches:** results from five detectors (SSH log anomalies, PPE compliance in camera images, face recognition, drive failure risk, room temperature and humidity).
- **What it adds:** it links results that happen close together in time and place, using a Neo4j knowledge graph, and gives each a severity from fixed rules.
- **Who decides:** a human operator reviews everything. The system never takes action on its own, and an optional AI Investigator answers questions with cited, approved knowledge.

All data is synthetic or public test data.

---

1. Requirements

	Minimum	Notes
Operating system	Windows 10 or 11	Tested on Windows 11. The commands in section 4 also work on Linux or macOS with `/` paths and `bin/` instead of `Scripts\` (not tested).
Memory and disk	16 GB RAM recommended (8 GB works for the demo); 15 GB free disk	The Python packages (TensorFlow, PyTorch) are large. Setup and every check also passed on a laptop with 7.4 GB RAM
GPU	**Not required**	With an NVIDIA GPU, PPE image checks are faster (optional `.venv-ppe`, section 7.1)
Git + **Git LFS**	Git 2.40+, Git LFS 3+	LFS stores the 318 MB drive-failure model. Install from https://git-lfs.com, then run `git lfs install` once
Python	**3.12**	https://www.python.org/downloads/; tick "Add python.exe to PATH"
Node.js	20 or newer	https://nodejs.org (LTS)
Neo4j	2026.08 (tested)	Neo4j Desktop (https://neo4j.com/download/) is easiest. Other recent versions and Community Server should work but were not tested.
OpenAI API key	Optional	For the AI Investigator and the specialist agents (section 7.3). **Without a key**, the Decisions for **SSH events** and for **drive + room-temperature pairs** are not made, because their rules need a specialist analysis first. Correlated groups and single PPE, face, drive and sensor results still get a rule-based severity
Internet	During setup	Packages, the embedding model for the knowledge index, and the face models DeepFace downloads on first use

After installing the prerequisites, **close and reopen PowerShell (or VS Code)**; otherwise `git` and `node` are "not recognized". Then check them before the long install:

```
git --version; git lfs version; py -3.12 --version; node --version
```

Each command should print a version (Git 2.40+, Git LFS 3+, Python 3.12, Node.js 20+).

2. Get the code

Use **git clone** where possible. GitHub's "Download ZIP" replaces the 318 MB Git LFS model with a small placeholder file. A ZIP that does contain the model (for example the course submission ZIP) also works: `setup.ps1` checks the model file and skips the Git steps when the folder is not a Git clone.

```
git lfs install
git clone https://github.com/Jowharah/DC-Guardian.git
cd DC-Guardian
git lfs pull
```

From the course submission repository instead, the code is in `SystemCode/`:

```
git clone https://github.com/Jowharah/IRS-PM-2026-10-25-AIS-GRP22-DC-Guardian-System.git
cd IRS-PM-2026-10-25-AIS-GRP22-DC-Guardian-System\SystemCode
git lfs pull
```

In the rest of this guide, "the project folder" is `DC-Guardian` or `SystemCode`. The setup script checks the drive-failure model file. If it is still a small text placeholder, run `git lfs pull` again.

3. Prepare Neo4j

1. Install and open **Neo4j Desktop**. Create a local instance, set a password you will remember, and **Start** it. The instance name can be anything; the database user is always `neo4j`.
2. The dashboard connects to `bolt://127.0.0.1:7687` with user `neo4j`. Use `127.0.0.1` rather than `localhost`: on Windows, `localhost` adds about 2 seconds per connection.
3. Keep Neo4j running whenever DC-GUARDIAN runs.

4. Install (one command)

In PowerShell, in the project folder:

```
powershell -ExecutionPolicy Bypass -File setup.ps1
```

It asks for your Neo4j password and a new dashboard password. It takes about 15–40 minutes, mostly package downloads (15 minutes on a test laptop with some packages already cached). It then:

1. checks Python 3.12, Node.js and, in a Git clone, Git LFS;
2. fetches the LFS model file (in a Git clone) and checks that it is the real model;
3. creates the Python environment `.venv-dc-guardian` from `requirements.txt`;

4. creates `.env` from `.env.example`: Neo4j settings, the dashboard login (user `DCG_LOCAL_USER`, `operator` by default; role administrator, zones A–C), AI Investigator off. An existing `.env` is kept;
5. installs the frontend packages (`npm ci`);
6. loads the DC-01 data-centre topology into Neo4j (zones, racks, servers, sensors, cameras, staff and their zone access);
7. verifies the 13 approved public knowledge documents (NIST, CISA, OSHA, U.S. DOE; bundled in the repository, see `response/rag/knowledge_base/bundled/README.md`) and builds the local search index for the RAG specialists;
8. loads the demo data (section 6).

Options:

- `-Gpu`: add the GPU environment for PPE.
- `-Face`: add a separate face environment.
- `-SkipRag`: no knowledge index; specialists then report knowledge as unavailable.
- `-SkipDemo`: no demo data.
- `-NeoPassword <pw> -DashboardPassword <pw>`: give both passwords instead of being asked, for an unattended run. They stay in the shell history, so the prompt is safer; creating `.env` yourself before running also works.

The script is safe to re-run.

Manual installation (other operating systems, or step by step)

```
py -3.12 -m venv .venv-dc-guardian
.venv-dc-guardian\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env        # then fill NEO4J_URI=bolt://127.0.0.1:7687, NEO4J_USER=neo4j,
                              # NEO4J_PASSWORD, NEO4J_DATABASE=neo4j, DCG_LOCAL_USER,
DCG_LOCAL_PASSWORD,
                              # DCG_LOCAL_ROLE=administrator, DCG_LOCAL_ZONES=ZONE-A,ZONE-
B,ZONE-C
cd presentation\frontend; npm ci; cd ..\..
$env:PYTHONPATH = (Get-Location).Path
New-Item -ItemType Directory -Force response\rag\knowledge_base\originals | Out-Null
Copy-Item response\rag\knowledge_base\bundled\* response\rag\knowledge_base\originals\ -Exclude
README.md   # approved documents ship with the code
.venv-dc-guardian\Scripts\python.exe reasoning\graph\load_topology.py
foreach ($s in
"fetch_sources","parse_sources","preprocess_sources","chunk_sources","build_index") { .venv-dc-
guardian\Scripts\python.exe response\rag\$s.py }
.venv-dc-guardian\Scripts\python.exe demo_data\load_demo_data.py
```

5. Start DC-GUARDIAN

Open two terminals in the project folder.

```
# Terminal 1: backend API
.venv-dc-guardian\Scripts\python.exe -m uvicorn presentation.backend.app.main:app --host
127.0.0.1 --port 8000

# Terminal 2: dashboard
cd presentation\frontend
npm run dev
```

Open **http://127.0.0.1:5173** and sign in with the user in `DCG_LOCAL_USER` in `.env` (`operator` unless you changed it) and the dashboard password you chose. To stop, press Ctrl+C in both terminals.

If ports 5173 or 8000 are taken, set `DCG_FRONTEND_PORT` and `DCG_BACKEND_URL` (for example `$env:DCG_BACKEND_URL="http://127.0.0.1:8010"`) before `npm run dev`, and start the backend on that port.

6. Demo data and first steps

`setup.ps1` loads the demo data (`demo_data/`, see its README): an SSH brute-force burst in ZONE-A (four public LogHub log lines), plus two drive histories and two room-sensor batches in ZONE-B, one rising above 35 °C. It contains no images. Re-run it any time with `.venv-dc-guardian\Scripts\python.exe demo_data\load_demo_data.py`; files already loaded are skipped.

A five-minute tour:

1. **Monitoring Center:** - The summary cards show what needs attention. - The domain counts read "N abnormal of M". *Abnormal* means a detector flagged something that is **not yet confirmed by a human**. - **How to read this screen**, under the cards, explains the main terms. - The event feed lists every result, newest first, colour-coded by domain. Filter it by date (**Received from … to …**), zone and severity.
2. **Open the Environmental + Maintenance correlation in ZONE-B:** - It links a drive-risk result and a room-temperature result by place and time. - It shows the Bayesian estimate of how likely the two are related. With an OpenAI key, run its specialist evaluation (section 7.3) to get its rule-based Decision and the rules that set it; without a key, the pair has no Decision. - Correlation is context, not proof of cause. - Three or more linked results form a **correlated group**; add a camera image in ZONE-B through Scenario Lab to see one. A group opens with an **At a glance** box: why grouped, **not proof of cause**, severity **set by fixed rules, not by AI** (with the caveat "validated on the author's labels only"), and the review status. **Simple view** shows the essentials; **Full detail** adds the graph, timeline, AI analysis and source links.
3. **Open a single result:** - An **At a glance** summary comes first. - Record your **human verdict** (Confirm / Override / Inconclusive, with a reason); a green **Verdict saved** box confirms it. - The detector's original output is never changed; your verdict is stored beside it. Inside a group, each event has its own **Record verdict** section.
4. **Investigation state:** on a group or result, press **Start review**, then **Resolve** with an outcome and a reason of at least 15 characters, and tick the confirmation.
5. **Human Review Queue:** - Work is sorted into **To do**, **In progress** and **Completed**, with a filter by type. - Each item carries a tag with its detailed state (for example "Not reviewed", "Group review needed", "Human override"). - Recording a verdict moves the item to Completed.
6. **Scenario Lab:** add your own data, for example a photo of a person in **PPE image validation** for a zone, or an OpenSSH log in **SSH log validation**. New results appear in the Monitoring Center within seconds.

7. Optional features

7.1 GPU for PPE

Run `setup.ps1 -Gpu`. It needs an NVIDIA GPU with a CUDA 12.6-compatible driver and creates `.venv-ppe` from `requirements-ppe.txt`. The backend uses it automatically when it exists. Without it, PPE runs on the CPU in the main environment: same model and policy, about 0.5–2 s per image.

7.2 Face recognition

Face recognition needs an **enrollment** of known staff faces, which is never included in the repository (biometric data).

1. **Add enrollment photos:** put front-facing photos, ideally several per person, into `evidence\face_recognition\data\controlled_faces\enrollment\P001\` … `P005\`. The project used public actor photos from the Kaggle *Face Verification Dataset* (MIT); see `docs/DATASETS_AND_CITATIONS.md`.
2. **Build the enrollment:** run `.venv-dc-guardian\Scripts\python.exe -m evidence.face_recognition.src.build_enrollment`.
3. **Grant zone access:** use **Scenario Lab → Administration & Test Utilities → Employee Access Management** on the dashboard.

Without an enrollment, a face upload answers "FACE_ENROLLMENT_MISSING"; everything else works.

7.3 AI Investigator and specialist agents (OpenAI)

The key enables the AI Investigator, the specialist panels, and the Decisions for SSH events and drive + room-temperature pairs (their rules use the specialist's analysis; the severity itself still comes only from the fixed rules).

1. Put your key in `.env` (`OPENAI_API_KEY=...`) and set `DCG_INVESTIGATOR_ENABLED=1`.
2. Restart the backend.

How it works when enabled:

- **Consent each time:** every question or specialist evaluation asks for consent first, then sends the selected **synthetic** Evidence to OpenAI. Usernames, IPs and person IDs are replaced by pseudonyms.
- **Limits:** at most 10 calls per minute per user.
- **No severity, no action:** answers cite approved knowledge, and a post-generation check flags unsupported claims. The AI never assigns severity or takes action.

8. Using the dashboard

Screen	What it is for
Monitoring Center	Live overview: summary cards (needs attention, abnormal results, correlations, rule-based Decisions by severity), domain counts ("N abnormal of M"), a **How to read this screen** guide, and the feed of all Evidence and correlated groups, colour-coded by domain, with **date range, zone and severity filters**. Selecting an item opens its detail with an **At a glance** summary first; groups open in a **Simple view**, with **Full detail** for the Neo4j graph (zone highlighted), timeline, AI analysis and source links.
Human Review Queue	Everything waiting for a human, in three tabs: **To do** (nothing recorded yet), **In progress** (follow-up open, or inconclusive) and **Completed** (verdict or review recorded), plus All and a type filter. Each item has a coloured tag with its detailed state. Open an item to record a verdict, or view it in the Monitoring Center.
Analytics Dashboard	Counts over a chosen period (24 hours, 7 days, 30 days, all, or a **custom period** from a calendar): Evidence by domain and by zone, detector outcomes with human verdicts, Decision severity, and **Review progress** (completed, in progress, to do per domain and for groups). Click a row to open that record.
Scenario Lab	Add data through the real detectors: PPE image validation, face recognition, SSH log validation, drive SMART history, sensor readings. It also runs built-in synthetic scenarios.
AI Investigator	Inside a group or result: ask questions about that Evidence (optional, section 7.3). Like the specialist panels, it carries a **Who says what** note: the detector produced the result, the AI only explains it, fixed rules set severity, and the human verdict is the review that counts.
Administration & Test Utilities (in Scenario Lab)	Employee access per zone (stored in Neo4j), audit-integrity checks, test utilities.

Rules the system follows (shown on screen):

- Severity comes only from fixed rules, never from the AI.
- Correlation means shared place and time, never proven cause or identity.
- Non-compliant PPE means "not detected", not "proven absent".
- Every result goes to human review, and nothing acts on its own.

9. Check the installation

```
.venv-dc-guardian\Scripts\python.exe check_all.py
```

It runs every test suite (about 5 minutes) and writes `logs\checks\<time>\report.md`. On a fresh installation:

- **Expected skips:** tests needing private data (face photos, training datasets) or a GPU environment skip with a stated reason.
- **Live API smoke test:** runs only while the backend is running.
- **No OpenAI calls:** the checks never call OpenAI. Specialist tests use recorded answers, so passing checks do not show that the Decisions needing a key (section 1) exist.

On the test laptop the checks took about 5 minutes: 12 passed, 0 failed, 2 skipped (GPU environments not installed).

10. Troubleshooting

Problem	Fix
Setup step 2: "is a small Git LFS placeholder", or a maintenance error	The drive-failure model was not downloaded. In a clone, run `git lfs install` and then `git lfs pull`. From GitHub's "Download ZIP", clone the repository instead (section 2)
"git is not recognized" (or `node`) right after installing it	Close and reopen PowerShell or VS Code (section 1)
A drive + room-temperature pair or an SSH event has no Decision; "Decision unavailable" (503)	Expected without an OpenAI key (section 1). Add a key (section 7.3) and run the specialist evaluation
Harmless setup messages	Expected, no action needed: step 8 "specialist unavailable … OPENAI_API_KEY is not configured" (the demo loader never sends data to OpenAI); npm "install-scripts … esbuild … not yet covered by allowScripts"; Hugging Face "unauthenticated requests to the HF Hub" and a symlink warning while building the index; pip "Connection interrupted … Attempting to resume"
"Neo4j not reachable"	Start the instance in Neo4j Desktop; check `NEO4J_PASSWORD` in `.env` and use `bolt://127.0.0.1:7687`
Dashboard shows "API unavailable"	The backend is not running, or it runs on another port (section 5)
PPE upload is slow	Normal on CPU (0.5–2 s); the first upload after starting also loads the model (5–15 s)
Face upload: FACE_ENROLLMENT_MISSING	Expected without an enrollment (section 7.2)
Specialists: "approved knowledge unavailable"	Run the RAG step (`setup.ps1` without `-SkipRag`); it downloads the small embedding model once
Setup step 7 shows `FAIL: ...` for an approved document	The 13 approved documents ship in `response/rag/knowledge_base/bundled` and are verified against `response/rag/manifests/source_lock.json`. A mismatch means a copy was changed: restore them (in a clone, `git checkout -- response/rag/knowledge_base/bundled response/rag/manifests/source_lock.json`; from a ZIP, extract those two folders again), delete the folder `response/rag/knowledge_base/originals`, and re-run `setup.ps1`
`npm` or `pip` certificate errors behind antivirus or corporate TLS inspection	Point Node at your organisation's root certificate: `setx NODE_EXTRA_CA_CERTS <path-to-root.pem>`, then open a new terminal. For pip, use `pip install --cert <path>`
Port already in use	See section 5 (`DCG_FRONTEND_PORT`, `DCG_BACKEND_URL`)
Logs	`logs\errors.log` (start here), `logs\backend.log`, `logs\api_access.log`; inference workers in `logs\workers\`

11. Data, privacy and licences

- **Data:** all data is synthetic or public test data. Never load real personal data.

- **Never committed:** `.env`, `logs/`, `presentation/backend/data/`, datasets, face photos and the enrollment.
- **Licences:** academic, non-commercial use. The PPE models are trained on SH17 (CC BY-NC-SA 4.0) with Ultralytics YOLOv8 (AGPL-3.0). See Datasets and Citations for every dataset, model and source, with citations.